VERIQTA

Linux TROUBLESHOOTING HANDBOOK

100 COMMANDS FOR REAL PRODUCTION FAILURES



@veriqta Instagram
@veriqta GitHub
@veriqta LinkedIn
@veriqta Telegram
@veriqta X
@veriqta Discord

VERIQTA

CPU at 100%, but Which Process Is Actually Responsible?



Production Scenario

- An API server experiences **severe latency** while CPU utilization reaches **100%**.
- Engineers suspect application traffic, but the real bottleneck is unclear.
- Use the commands below to identify which process, thread or system activity is consuming CPU and why.

1. uptime – Check Load Averages and Uptime

```
uptime
```

- Shows system uptime, number of users and load averages (1, 5 and 15 minutes).
- Load average indicates runnable and uninterruptible tasks, not just CPU usage.

```
[root@server ~]# uptime
 15:42:01 up 12 days,  3:21,  2 users,  load average:
32.41, 28.17, 20.03
```

2. mpstat -P ALL 1 5 – Check CPU Usage per Core

```
mpstat -P ALL 1 5
```

- Shows CPU usage per core every second for 5 seconds.
- Identify high user time (application), system time (kernel) and steal time (VM contention).

```
[root@server ~]# mpstat -P ALL 1 5
Linux 5.15.0 (server)     10/22/2026     _x86_64_     (8 CPU)

          %usr     %sys   %iowait    %steal     %idle
all      72.34    18.12      2.11      0.00      7.43
  0      98.11     1.23      0.12      0.00      0.54
  1      96.44     2.01      0.08      0.00      1.47
  2      12.31     4.21      1.11      0.00     82.36
```

3. pidstat -u 1 5 – Find Processes Using CPU

```
pidstat -u 1 5
```

- Shows per-process CPU usage every second for 5 seconds.
- Helps identify which processes are generating the load.

```
[root@server ~]# pidstat -u 1 5
UID       PID    %usr %system  %CPU   Command
root     1234   85.12   12.21 97.33   java
root     5678    1.21    0.33  1.54   nginx
root     9012    0.12    0.21  0.33   python
mysql    3456    0.45    0.11  0.56   mysqld
root     7890    0.02    0.01  0.03   kworker
```

4. top -H -p <PID> – Investigate CPU-Intensive Threads

```
top -H -p 1234
```

- Shows individual threads for a specific process.
- Helps identify which thread(s) are consumiong CPU.

```
top - 15:42:36 up 12 days,  3:21,  2 users,  load average: 32.4
Threads: 120 total,   5 running, 115 sleeping,   0 stopped,   0
  PID  USER      PR  NI    VIRT    RES  %CPU  COMMAND
 1234  root      20   0    2.1g   512m 398.5  java
 1240  root      20   0    2.1g   512m 125.3  java
 1241  root      20   0    2.1g   512m  98.7  java
 1245  root      20   0    2.1g   512m  12.1  java
 1247  root      20   0    2.1g   512m   8.4  java
```

5. ps -eo pid,ppid,stat,pcpu,comm --sort=-pcpu

```
ps -eo pid,ppid,stat,pcpu,comm --sort=-pcpu
```

- Lists processes sorted by CPU usage.
- Shows parent process (PPID) and state to understand hierarchy.

```
[root@server ~]# ps -eo pid,ppid,stat,pcpu,comm --sort=-pcpu
  PID    PPID  STAT   %CPU  COMMAND
 1234       1  R      97.6  java
 5678       1  S       2.1  nginx
 9012    5678  S       1.3  worker
 3456       1  S       0.9  mysqld
 7890       1  S       0.4  python
```

6. How to Interpret the Results

Observation	Possible Cause
High %usr	Application code consuming CPU
High %sys	Kernel activity, system calls, driver issues
High %steal	Virtual machine CPU contention
High load average, low CPU	Blocked tasks (I/O, network, locks)
Single process or thread high	Application issue or runaway process
All cores high	High traffic or inefficient code
Intermittent spikes	Cron jobs, backups, GC, or external dependency

7. Senior Engineering Focus

- Distinguish CPU saturation from high load caused by blocked tasks.
- Determine whether the problem involves application code, kernel activity, CPU quotas, or virtual machine contention.
- Correlate process, thread and system metrics before taking action.
- Investigate recent changes, deployments, or traffic spikes.

8. Key Takeaways

- High CPU does not always mean application traffic.
- Use multiple commands to confirm the real cause.
- Check per-core usage, process and thread level details.
- Consider kernel activity, I/O waits, or VM contention.
- Take evidence-based action, not assumptions.

VERIQTA

The Server Has Free Memory, but Linux Is Killing Processes



Production Scenario

- A Java service is terminated unexpectedly despite monitoring showing available memory.
- The server appears to have free memory, but Linux kills the process.
- Use the commands below to investigate memory usage, OOM events and cgroup limits.

1. free -h – Inspect Available Memory, Cache and Swap

```
free -h
```

- Shows total, used, free, cached and available memory.
- Helps understand memory usage at a high level.

```
[root@server ~]# free -h
               total        used        free      shared  buff/cache   available
Mem:            31G         18G        3.2G        512M        9.8G         11G
Swap:          8.0G        1.2G        6.8G
```

2. cat /proc/meminfo – Check Detailed Memory Info

```
cat /proc/meminfo
```

- Shows detailed kernel memory accounting.
- Check MemFree, MemAvailable, Cached, SReclaimable and Swap.

```
[root@server ~]# cat /proc/meminfo | egrep 'MemFree|MemAvailable|Cached|SReclaimable|Swap'
MemFree:         3456788 kB
MemAvailable:   11234567 kB
Cached:          6789012 kB
SReclaimable:    2345678 kB
SwapTotal:       8388608 kB
SwapFree:        7134560 kB
```

3. vmstat 1 5 – Observe Memory Pressure and Swapping

```
vmstat 1 5
```

- Shows memory, swapping, I/O and blocked tasks every second.
- High values in si/so indicate swapping. High wa indicates I/O wait.

```
[root@server ~]# vmstat 1 5
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa
 5  0   1024 3254688 234512 9876540  120  340    12   320 1045 2301 15 25 50 10
 8  0   2048 3124480 234660 9865432  250  610    22   540 1100 2501 18 30 40 12
 6  0   2048 3102240 234760 9854320  300  780    18   610 1200 2601 20 28 38 14
 4  0   2048 3081120 234800 9843210  280  690    20   500 1180 2550 17 26 41 16
 3  0   2048 3069980 234820 9832100  260  640    10   420 1150 2480 16 24 46 14
```

4. Check Kernel Logs for OOM Events

```
journalctl -k -g 'oom|Out of memory|Killed process' --no-pager
```

- Searches kernel logs for out-of-memory events.
- Shows which process was killed and why.

```
[root@server ~]# journalctl -k -g 'oom|Out of memory|Killed process' --no-pager
Oct 22 14:32:11 server kernel: Out of memory: Killed process 12345 (java) total-vm:8123456kB, anon-rss:6234567kB, file-rss:0kB, shmem-rss:0kB, UID:1000 pgtables:12345kB oom_score_adj:0
Oct 22 14:32:11 server kernel: oom_reaper: reaped process 12345 (java), now anon-rss:0kB, file-rss:0kB
```

5. Check cgroup v2 Memory Events

```
cat /sys/fs/cgroup/memory.events
```

- Shows memory events for the current cgroup (cgroup v2).
- Helps identify if the process was killed due to cgroup memory limits.

```
[root@server ~]# cat /sys/fs/cgroup/memory.events
low                0
high               15
max                3
oom                2
oom_kill           2
oom_group_kill     0
```

6. Common Causes and What They Mean

Indicator	What It Means	Likely Cause
High swap usage (si/so)	System is swapping memory	Memory pressure
OOM in kernel logs	Kernel killed a process	Global OOM or cgroup limit
oom_kill > 0 in cgroup	Process killed by cgroup	Container memory limit
High cached memory	Memory used for cache (reclaimable)	Normal Linux behaviour
MemAvailable low	Little memory available for new allocations	Memory pressure
Process killed but free memory exists	Cgroup limit or memory overcommit	Container limit or per-process limit

7. Senior Engineering Focus

- Differentiate global OOM events from cgroup memory limits.
- Understand why free host memory does not guarantee a container can allocate more memory.
- Check application memory usage, JVM settings, memory leaks and memory limits.
- Determine if the issue is application code, kernel behaviour, container limits or resource contention.

8. Key Takeaways

- Free memory does not always mean memory is available for new processes.
- Check kernel logs and cgroup events for the real cause.
- Monitor swapping, memory pressure and container limits.
- Tune application memory settings and container resource limits.
- Investigate memory leaks and adjust JVM/container configurations.

VERIQTA

Load Average Is 80, but CPU Utilization Is Only 15%



Production Scenario

- A Linux database server reports **extreme load**, yet most CPU cores appear idle.
- Users experience slow queries and high response times.
- The load average is 80, but CPU utilization is only 15%.
- Use the commands below to find out why and identify the real bottleneck.

1. cat /proc/loadavg – Check System Load

```
cat /proc/loadavg
```

- Shows 1, 5 and 15 minute load averages.
- Also shows the number of runnable and total tasks.

```
[root@server ~]# cat /proc/loadavg
82.15   79.83   67.42   12/582   33421
```

1 min load average | 5 min load average | 15 min load average | Runnable/Total tasks | Last PID

2. vmstat 1 5 – Check Runnable and Blocked Tasks

```
vmstat 1 5
```

- Shows memory, swapping, I/O and blocked tasks every second.
- High values in r (runnable) or b (blocked) indicate issues.

```
[root@server ~]# vmstat 1 5
procs -----------memory------- ---swap-- -----io--- -system-- ---cpu---
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa
72 18    0 421320 82344 312904    0    0  1248  9864 3212  498  6 8 15 71
68 21    0 420112 82520 312816    0    0  1310 10234 3120  480  5 7 16 72
75 20    0 419848 82420 312900    0    0  1402 11021 3180  510  4 6 14 76
70 19    0 419932 82510 312876    0    0  1298 10524 3091  488  6 5 15 74
```

3. ps -eo pid,stat,wchan:32,comm – Find Blocked Processes

```
ps -eo pid,stat,wchan:32,comm
```

- Shows process state and the kernel function it is waiting on.
- Processes in state "D" are in uninterruptible sleep (usually I/O).

```
[root@server ~]# ps -eo pid,stat,wchan:32,comm | head
  PID    STAT    WCHAN                            COMMAND
 1234    D       io_schedule                      mysqld
 5678    D       blk_mq_get_request               mysqld
 9012    D       do_wait                          postgres
 3456    D       __xfs_filemap_fault              java
 7890    S       ep_poll_wait                     nginx
 1122    R       -                                top
```

4. cat /proc/pressure/io – Check I/O Pressure

```
cat /proc/pressure/io
```

- Shows I/O pressure stall information (PSI).
- High values indicate the system is spending time waiting for I/O.

```
[root@server ~]# cat /proc/pressure/io
some avg10=0.62 avg60=0.48 avg300=0.31 total=123456789
full avg10=0.41 avg60=0.29 avg300=0.18 total=98765432
```

How to Read It

- some: % of time at least one task is waiting for I/O.
- full: % of time all non-idle tasks are waiting for I/O.
- High values (e.g. > 0.20) indicate significant I/O contention.

5. iostat -xz 1 5 – Check Disk Latency and Utilization

```
iostat -xz 1 5
```

- Shows detailed block device statistics every second.
- High await, high %util or high queue depth indicates storage issues.

```
[root@server ~]# iostat -xz 1 5
Device      r/s   w/s   rkB/s   wkB/s   avgrq-sz  avgqu-sz  await  r_await  w_await  %util
sda        12.1   3.4    2456    1024      256.3      24.5  180.2     12.3    240.1   98.7
nvme0n1     1.2   0.8     120      40      128.1       0.6    2.3      0.4      3.2    4.1
```

6. Why Load Can Be High with Low CPU Usage

High Load Average

- Runnable tasks (R)
- Blocked tasks (D)
- Waiting for I/O
- Waiting for locks
- Waiting for network
- Kernel operations

→ **Linux Load Average =**

Runnable tasks + Uninterruptible sleep + Other waiting tasks

→ **Low CPU Usage**

- CPU cores are mostly idle
- Tasks are waiting (not using CPU)
- Usually caused by storage, network, locks or external dependencies

7. Senior Engineering Focus

- Understand that load average includes tasks waiting in uninterruptible sleep (D).
- Distinguish between CPU saturation and I/O or other resource bottlenecks.
- Use multiple commands (loadavg, vmstat, iostat, PSI) to correlate evidence.
- Investigate storage performance, I/O latency, filesystem issues and database activity.
- Determine the real bottleneck before taking action.

8. Key Takeaways

- High load average does not always mean high CPU usage.
- Tasks in uninterruptible sleep (D) increase load average.
- Storage problems can create high load without CPU saturation.
- Use ps, iostat, vmstat and PSI to identify the real cause.
- Fix the underlying I/O, storage or dependency issue, not just scale CPUs.

VERIQTA

A Linux Service Keeps Restarting Without an Obvious Error



Production Scenario

- A critical API service repeatedly restarts under systemd, creating intermittent outages.
- The service appears to start, then stops shortly after.
- Logs do not show a clear error message.
- Use the commands below to investigate the real cause.

1. systemctl status api.service – Check Current Status

```
systemctl status api.service
```

- Shows current service state, recent failures and exit codes.
- Helps identify if the service is failing, restarting or inactive.

```
[root@server ~]# systemctl status api.service
● api.service - API Service
     Loaded: loaded (/etc/systemd/system/api.service; enabled;
     Active: activating (auto-restart) (Result: exit-code)
       Docs: https://example.com
    Process: 12345 ExecStart=/usr/bin/java -jar api.jar
             (code=exited, status=1/FAILURE)
   Main PID: 12345 (code=exited, status=1/FAILURE)
    Restart: always
RestartSec: 5s
        CPU: 2.314s
```

2. journalctl -u api.service -b --no-pager – Review Logs

```
journalctl -u api.service -b --no-pager
```

- Shows logs for the service from the current boot.
- Helps identify the reason for crashes or failures.

```
[root@server ~]# journalctl -u api-service -b --no-pager
Oct 22 14:32:10 server systemd[1]: Started API Service.
Oct 22 14:32:12 server api[12345]: Exception in thread
"main" java.lang.NullPointerException
Oct 22 14:32:12 server api[12345]: at
com.example.App.main(App.java:25)
Oct 22 14:32:12 server systemd[1]: api.service:
Main process exited, code=exited, status=1/FAILURE
Oct 22 14:32:12 server systemd[1]: api.service: Failed
with result 'exit-code'.
Oct 22 14:32:17 server systemd[1]: api.service: Scheduled
restart job, restart counter is at 3.
```

3. systemctl show api.service – Check Restart Policy and Results

```
systemctl show api.service -p Restart -p NRestarts \\
-p ExecMainStatus -p Result
```

- Shows restart policy, number of restarts, exit status and result.
- Helps understand why the service keeps restarting.

```
[root@server ~]# systemctl show api.service -p Restart \
-p NRestarts -p ExecMainStatus -p Result
Restart=always
NRestarts=5
ExecMainStatus=1
Result=exit-code
```

4. systemctl cat api.service – Check Unit File and Drop-ins

```
systemctl cat api.service
```

- Shows the full unit file and any drop-in configuration.
- Helps identify misconfiguration, wrong paths or environment issues.

```
[root@server ~]# systemctl cat api.service
# /etc/systemd/system/api.service
[Unit]
Description=API Service
After=network.target
[Service]
ExecStart=/usr/bin/java -jar /opt/api/api.jar
Restart=always
RestartSec=5
Environment=JAVA_OPTS=-Xmx512m
[Install]
WantedBy=multi-user.target
```

5. systemd-analyze critical-chain api.service – Check Dependencies

```
systemd-analyze critical-chain api.service
```

- Shows the startup ordering and time taken by dependencies.
- Helps identify slow or failed dependencies delaying the service.

```
[root@server ~]# systemd-analyze critical-chain api.service
The time after the unit is active or started is printed after the "@".
The time the unit takes to start is printed after the "+".

api.service @5.234s
└─network.target @5.120s
  └─systemd-networkd.service @3.245s +1.874s
    └─basic.target @3.200s
      └─sockets.target @3.200s
```

6. Senior Engineering Focus

- Distinguish application crashes, watchdog failures, systemd timeouts, dependency failures and restart-policy problems.
- Do not assume it is an application bug without checking systemd configuration and dependencies.
- Review recent changes, environment variables, resource limits and logs.
- Avoid hiding a recurring fault with unlimited automatic restarts.

7. Key Takeaways

- A repeatedly restarting service usually has a specific reason.
- Always check logs, exit codes and restart policy.
- Investigate dependencies and environment configuration.
- Fix the root cause instead of increasing restart limits.
- Monitoring should alert on repeated restarts.

VERIQTA

Disk Usage Shows 100%, but Nobody Can Find the Files



Production Scenario

- A production server **cannot write logs or temporary files** even after engineers delete large files.
- Disk usage shows 100% but there are no obvious large files.
- Use the commands below to find the real cause.

1. df -hT – Inspect Filesystem Capacity and Type

`df -hT`

- Shows filesystem size, used, available space and type.
- Helps identify which filesystem is full.

```
[root@server ~]# df -hT
Filesystem     Type      Size  Used Avail Use% Mounted on
/dev/sda1      xfs        50G   50G     0  100% /
tmpfs          tmpfs     1.9G     0  1.9G    0% /dev/shm
/dev/sdb1      xfs       200G  120G   80G   60% /data
/dev/sdc1      xfs       100G   99G    1G   99% /var
```

2. du -xhd1 /var – Find Large Directories

`du -xhd1 /var`

- Shows size of directories in /var without crossing filesystems.
- Helps identify which directory is consuming space.

```
[root@server ~]# du -xhd1 /var
45G     /var/log
8.1G    /var/lib
2.3G    /var/cache
1.1G    /var/spool
512M    /var/tmp
60G     /var
```

3. lsof +L1 – Find Deleted Files Still Held Open

`lsof +L1`

- Lists files that have been deleted but are still open by processes.
- Explains why disk space remains used even after deletion.

```
[root@server ~]# lsof +L1
COMMAND   PID  USER   FD   TYPE  SIZE/OFF  NLINK  NAME
java      1234 app    255w REG     20G      0    /var/log/app.log (deleted)
nginx     5678 www    123w REG     5.2G     0    /var/log/nginx/access.log (deleted)
python    9012 app     87w REG     1.1G     0    /tmp/data.tmp (deleted)
```

4. df -i – Check for Inode Exhaustion

`df -i`

- Shows inode usage instead of disk space.
- Helps detect inode exhaustion (no space for new files).

```
[root@server ~]# df -i
Filesystem     Inodes   IUsed   IFree IUse% Mounted on
/dev/sda1    3276800 3276780      20  100% /
tmpfs         483840     320  483520    1% /dev/shm
/dev/sdb1   13107200 6543210 6563990   50% /data
/dev/sdc1    6553600 6553000     600  100% /var
```

5. findmnt -T /var/log – Identify the Filesystem

`findmnt -T /var/log`

- Shows the filesystem, source device and mount options for a specific path.
- Helps detect if /var/log is a separate mount or a bind mount.

```
[root@server ~]# findmnt -T /var/log
TARGET    SOURCE     FSTYPE  OPTIONS
/var/log  /dev/sdc1  xfs     rw,relatime,attr2,inode64,noquota
```

Why Deleting a File May Not Free Disk Space

- File is still open by a running process (shown by lsof +L1).
- Data is written to an unlinked file (deleted but open).
- You are looking at the wrong filesystem or mount point.
- Disk space is consumed by inodes, not file size.
- Directory contains many small files (inode exhaustion).
- File is on a hidden mount, bind mount or separate filesystem.

6. Senior Engineering Focus

- Investigate deleted-but-open files.
- Check for inode exhaustion.
- Understand filesystem mount relationships and hidden mounts.
- Analyze filesystem allocation and usage patterns.
- Explain why deleting a file may not immediately release its disk space.
- Determine the real cause before taking action.

7. Key Takeaways

- Disk usage can remain high due to open deleted files.
- Always check both disk space and inode usage.
- Identify the exact filesystem and mount point.
- Look for hidden mounts, bind mounts and separate filesystems.
- Use multiple commands together to find the real cause.
- Do not assume deletion immediately frees disk space.

VERIQTA

Disk Latency Is Destroying Application Performance



Production Scenario

- Database requests become slow during peak traffic, but CPU and memory metrics appear normal.
- Use the commands below to investigate disk latency and identify the real cause.

26. iostat -xz 1 10 – Check Disk Latency, Queueing and Utilization

```
iostat -xz 1 10
```

- Shows device utilization, I/O latency, queue depth and throughput.
- Helps identify whether storage is the bottleneck.

```
[root@server ~]# iostat -xz 1 3
Device    r/s     w/s     rkB/s     wkB/s   await  r_await  w_await   %util
sda       0.2    45.6      12.4    1024.3    35.2      1.1     38.4    98.7
sdb       0.0     0.1       0.0       1.2     0.4      0.1      0.6     2.1
nvme0n1  10.1   220.3     512.6    4096.5    25.6      3.2     28.1    92.4
```

27. pidstat -d 1 5 – Find Processes Generating Disk I/O

```
pidstat -d 1 5
```

- Shows per-process disk I/O every second for 5 seconds.
- Helps identify which processes are causing high I/O.

```
[root@server ~]#  pidstat -d 1 3
  PID      kB_read/s    kB_wrtn/s   kB_ccwr/s    Command
 1234        1024.5       2048.6          0.0    mysqld
 5678          12.3         88.1          0.0    java
 9012           0.0          0.0          0.0    python
 3456         512.0       1024.0          0.0    postgres
```

28. iotop -oPa – Check Real-Time and Accumulated I/O

```
iotop -oPa
```

- Shows processes with active or accumulated disk I/O.
- Helps identify heavy I/O consumers in real time.

```
[root@server ~]# iotop -oPa | head
PID   USER      DISK READ   DISK WRITE   SWAPIN     IO>    COMMAND
1234  mysql      1.20 M/s    10.50 M/s   0.00 %   85.43 %  mysqld
5678  app       10.00 K/s   420.00 K/s   0.00 %   12.34 %  java
9012  root       0.00 B/s   120.00 K/s   0.00 %    5.21 %  backup
3456  postgres 512.00 K/s     1.10 M/s   0.00 %   25.12 %  postgres
```

29. cat /proc/diskstats – Check Kernel Block-Device Counters

```
cat /proc/diskstats
```

- Shows kernel block device counters.
- Helps analyze I/O patterns and detect abnormal activity.

```
[root@server ~]# cat /proc/diskstats | head -3
8   0 sda     157698 1024 1234567 23456  987654 2048 7654321 34567 0 0 0
8  16 sdb       2345   12   34567   890   12345   67   89012   345 0 0 0
259 0 nvme0n1  987654 5678 12345678 90123 23456789 6789 34567890 123456 0 0 0
```

30. lsblk -o NAME,TYPE,SIZE,FSTYPE,MOUNTPOINTS – Map Block Devices

```
lsblk -o NAME,TYPE,SIZE,FSTYPE,MOUNTPOINTS
```

- Shows kerck devices, size, filesystem type and mount points.
- Helps understand storage layout and which device is backing an application.

```
[root@server ~]# lsblk -o NAME,TYPE,SIZE,FSTYPE,MOUNTPOINTS
NAME        TYPE   SIZE   FSTYPE   MOUNTPOINTS
sda         disk   500G
├─sda1      part   100G   xfs      /
└─sda2      part   200G   xfs      /var
sdb         disk     1T   xfs      /data
nvme0n1     disk     1T   ext4     /backup
```

Senior Engineering Focus

- Distinguish storage latency from application processing time.
- Investigate workload contention, device limits and queueing.
- Check per-process I/O, device latency and filesystem activity.
- Correlate with application behaviour before recommending faster disks.

VERIQTA

Filesystem Errors Appear After a Storage Incident



Production Scenario

- A server recovers from an unexpected shutdown, but applications begin reporting read-only filesystem errors.
- Use the commands below to investigate the filesystem and storage health.

1. findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS – Check Mount Options

```
findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS
```

- Shows current mounts, filesystem types and options.
- Helps identify read-only mounts or unexpected mount configurations.

```
[root@server ~]# findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS
TARGET      SOURCE           FSTYPE  OPTIONS
/           /dev/sda1        xfs     rw,relatime,attr2,inode64
/var        /dev/sda2        xfs     ro,relatime,attr2,inode64
/data       /dev/mapper/data ext4    rw,relatime
```

2. journalctl -k -b – Check Kernel Disk and Filesystem Errors

```
journalctl -k -b --no-pager
```

- Shows kernel messages from the current boot.
- Helps identify filesystem errors, I/O errors and read-only remounts.

```
[root@server ~]# journalctl -k -b --no-pager | grep -iE 'error|ext4|xfs|readonly'
Oct 22 10:12:31 server kernel: XFS (sda2): Metadata corruption detected
Oct 22 10:12:31 server kernel: XFS (sda2): forcing filesystem read-only
Oct 22 10:12:32 server kernel: Buffer I/O error on dev sda2, logical block 123456
```

3. lsblk -f – Check Filesystems, UUIDs and Mountpoints

```
lsblk -f
```

- Shows filesystem types, UUIDs and mount points.
- Helps verify if the correct device is mounted and the filesystem type.

```
[root@server ~]# lsblk -f
NAME        FSTYPE          UUID                                 MOUNTPOINT
sda
├─sda1      xfs             a1b2-c3d4                            /
└─sda2      xfs             d4e5-f6a7                            /var
sdb         ext4            e7f8-a9b0                            /data
```

4. smartctl -a /dev/sdX – Check Device Health (If Available)

```
smartctl -a /dev/sda
```

- Shows SMART data for the storage device.
- Helps detect hardware issues.

```
[root@server ~]# smartctl -a /dev/sda | head
=== START OF INFORMATION SECTION ===
Model Family:  Samsung SSD 870
Serial Number: S6P1NB0R123456X
SMART overall-health self-assessment test result: PASSED
```

5. xfs_info /mountpoint – Examine XFS Filesystem (If Applicable)

```
xfs_info /var
```

- Shows XFS filesystem geometry and information.
- Helps validate the filesystem and detect inconsistencies.

```
[root@server ~]# xfs_info /var
meta-data=/dev/sda2      isize=512    agcount=4, agsize=13107200 blks
         =               sectsz=512   attr=2, projid32bit=1
data     =               bsize=4096   blocks=52428800, imaxpct=25
naming   =version 2      bsize=4096   ascii-ci=0 ftype=1
```

Senior Engineering Focus

- Investigate whether the filesystem was remounted read-only due to errors.
- Check kernel logs, device health and filesystem type.
- Confirm the correct device and mount options.
- Determine if the issue is hardware, filesystem corruption or unexpected shutdown.

8. Key Takeaways

- Check mount options for read-only state.
- Review kernel logs for filesystem errors.
- Verify device health and filesystem type.
- Do not run filesystem repair blindly.
- Identify the root cause before taking action.

VERIQTA

Filesystem Errors Appear After a Storage Incident



Production Scenario

- A server recovers from an unexpected shutdown, but applications begin reporting read-only filesystem errors.
- The filesystem may have been remounted as read-only due to errors.
- Use the commands below to investigate the filesystem, device and storage health.

31. findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS – Check Mount Options

```
findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS
```

- Shows current mount points, source devices, filesystem types and options.
- Helps identify if a filesystem has been remounted as read-only.

```
[root@server ~]# findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS | grep -E "/var|/"
TARGET     SOURCE      FSTYPE     OPTIONS
/          /dev/sda1   xfs        rw,relatime,attr2,inode64
/boot      /dev/sda2   xfs        rw,relatime
/var       /dev/sdb1   xfs        ro,relatime      # read-only
/var/log   /dev/sdb1   xfs        ro,relatime
```

32. journalctl -k -b – Check Kernel Storage and Filesystem Errors

```
journalctl -k -b --no-pager
```

- Shows kernel messages from the current boot.
- Helps identify filesystem errors, I/O errors and read-only remounts.

```
[root@server ~]# journalctl -k -b --no-pager | tail -n 10
Oct 22 10:14:21 server kernel: sd 0:0:0:0: [sda] I/O error
Oct 22 10:14:21 server kernel: Buffer I/O error on dev sda1
Oct 22 10:14:21 server kernel: XFS (sda1): metadata I/O error
Oct 22 10:14:21 server kernel: XFS (sda1): xfs_do_force_shutdown(
Oct 22 10:14:21 server kernel: XFS (sda1): Filesystem has been
Oct 22 10:14:21 server kernel: XFS (sda1): ending cleanly
Oct 22 10:14:21 server kernel: XFS (sda1): remounting filesystem
Oct 22 10:14:21 server kernel: XFS (sda1): read-only
```

33. lsblk -f – Map Filesystems, UUIDs and Mountpoints

```
lsblk -f
```

- Shows block devices, filesystem types, labels, UUIDs and mount points.
- Helps verify the correct device is mounted and the filesystem type.

```
[root@server ~]# lsblk -f
NAME   FSTYPE  LABEL   UUID                                   MOUNTPOINTS
sda
├─sda1 xfs     root    4f2a9f8c-1c3d-4e5b-8a2f-1d2e3f4a5b6c   /
├─sda2 xfs     boot    9e1a7b6c-2d3f-4c5e-8a7f-5d6e1f2a3b4c   /boot
sdb
├─sdb1 xfs     data    3a2b1c4d-5e6f-7d8a-9b0c-1d2e3f4a5b6d   /var
└─sdb2 ext4    backup  6c5d4e3f-2a1b-9c8d-7e6f-5a4b3c2d1e0f   /backup
```

34. smartctl -a /dev/sdX – Check Device Health (If Available)

```
smartctl -a /dev/sdX
```

- Shows SMART data for the storage device.
- Helps detect hardware issues such as reallocated sectors, read errors or a failing disk.

```
[root@server ~]# smartctl -a /dev/sda | head -20
=== START OF INFORMATION SECTION ===
Model Family:  Samsung SSD 870
Device Model:  Samsung SSD 870 EVO 1TB
Serial Number:  S6P9N0R123456X
SMART overall-health self-assessment test result: PASSED

ID#  ATTRIBUTE_NAME          FLAG     VALUE  WORST  THRESH  TYPE
  5  Reallocated_Sector_Ct   0x0033   100    100    10      Pre-fail
  9  Power_On_Hours          0x0032   98     98     0       Old_age
194  Temperature_Celsius     0x0022   70     50     0       Old_age
197  Current_Pending_Sector  0x0012   100    100    0       Old_age
198  Offline_Uncorrectable   0x0010   100    100    0       Old_age
```

35. xfs_info /mountpoint – Check XFS Filesystem Geometry (If XFS)

```
xfs_info /mountpoint
```

- Shows XFS filesystem geometry and key information.
- Helps validate the filesystem and detect inconsistencies.

```
[root@server ~]# xfs_info /var
meta-data=/dev/sdb1      isize=512    agcount=4, agsize=1310720 blks
         =               sectsz=512   attr=2, projid32bit=1
data     =               bsize=4096   blocks=5242880, imaxpct=25
         =               swidth=0     blks
naming   =version 2      bsize=4096   ascii-ci=0 ftype=1
log      =internal       bsize=4096   blocks=2560, version=2
         =               sectsz=512   sunit=0 blks, lazy-count=1
realtime =none           extsz=4096   blocks=0, rtextents=0
```

Senior Engineering Focus

- Determine whether the problem originates from hardware, block storage, filesystem corruption, or protective read-only remounting.
- Investigate kernel logs, device health, mount options and filesystem state together.
- Distinguish between transient I/O errors and real filesystem damage.
- Avoid unnecessary filesystem changes before confirming the root cause.

Important Operational Warning

- Never run filesystem repair blindly against a mounted production filesystem.
- Establish backups, filesystem type and recovery requirements first.
- Understand the impact and downtime before performing any repair.
- If the filesystem is read-only, investigate the cause instead of remounting read-write immediately.

VERIQTA

The Server Is Running Out of File Descriptors



Production Scenario

- A high-traffic API intermittently fails to accept connections and reports "Too many open files."
- New connections are rejected even though CPU and memory look normal.
- Use the commands below to investigate the real cause.

36. cat /proc/sys/fs/file-nr – Check System-Wide File Handles

```
cat /proc/sys/fs/file-nr
```

- Shows allocated, unused and maximum file handles.
- Helps determine if the system is close to the global limit.

```
[root@server ~]# cat /proc/sys/fs/file-nr
3210456     125432     4194304
```

Allocated file handles | Unused file handles | Maximum file handles

37. cat /proc/<PID>/limits – Check Process Open-File Limits

```
cat /proc/1234/limits
```

- Shows resource limits for a specific process.
- Helps verify the process's open file descriptor limit (NOFILE).

```
[root@server ~]# cat /proc/1234/limits | grep "open files"
Max open files            1024                 4096                 files
```

Soft limit (current) | Hard limit (maximum)

38. ls /proc/<PID>/fd | wc -l – Count Open Descriptors

```
ls /proc/1234/fd | wc -l
```

- Counts the number of open file descriptors for a process.
- Helps identify processes with unexpectedly high descriptor usage.

```
[root@server ~]# ls /proc/1234/fd | wc -l
2867
```

```
[root@server ~]# ls /proc/1234/fd | head
0
1
2
9
```

39. lsof -p <PID> – Identify Open Files and Sockets

```
lsof -p 1234 | head
```

- Lists open files, sockets and other descriptors for a process.
- Helps identify file, network and other resource usage.

```
[root@server ~]# lsof -p 1234 | head
COMMAND  PID  USER  FD   TYPE  DEVICE  SIZE/OFF  NODE  NAME
java     1234 app   cwd  DIR   253,0     4096     2  /opt/app
java     1234 app   rtd  DIR   253,0     4096     2  /
java     1234 app   txt  REG   253,0      123M   131  /usr/bin/java
java     1234 app   mem  REG   253,0       45M  9876  /lib/libjvm.so
java     1234 app   3u   IPv4  12345       0t0        TCP
java     1234 app   4u   IPv4  12346       0t0        TCP
```

40. ss -s – Summarize Socket Activity

```
ss -s
```

- Shows a summary of socket usage.
- Helps identify connection growth and potential descriptor exhaustion.

```
[root@server ~]# ss -s
Total: 5487
TCP:   4210 (estab 3987, closed 120, orphaned 0, timewait 103)
UDP:   45 (unconnected 40, connected 5)
RAW:   0
FRAG:  0
UNIX:  1232 (stream 1002, dgram 200, seqpacket 30, other 0)
```

Senior Engineering Focus

- Separate descriptor leaks from legitimate connection growth.
- Investigate which process or component is consuming file descriptors.
- Check both system-wide and per-process limits.
- Explain why increasing limits without fixing leaks. can postpone rather than solve the incident.

Key Takeaways

- Monitor file descriptor usage for critical services.
- Check both system and process limits.
- Identify if the increase is due to normal load or a leak.
- Look for unclosed files, sockets and other resources.
- Fix the root cause instead of only increasing limits.

VERIQTA

The Application Is Running, but Clients Cannot Connect



Production Scenario

- A service is active and listening, but external clients receive connection failures.
- The application appears healthy on the server.
- Use the commands below to investigate the real cause.

41. ss -lntp – Check Listening Sockets and Processes

```
ss -lntp
```

- Shows TCP listening sockets and the processes that own them.
- Helps verify if the service is listening on the expected address and port.

```
[root@server ~]# ss -lntp
State   Recv-Q  Send-Q  Local Address:Port   Peer Address   Process
LISTEN  0       4096    127.0.0.1:8080       0.0.0.0:*      users:("java",pid=1234)
LISTEN  0       4096      0.0.0.0:443        0.0.0.0:*      users:("nginx",pid=5678)
LISTEN  0       4096        [::]:22          [::]:*         users:("sshd",pid=987)
```

42. ip -br addr – Check Interface Addresses and States

```
ip -br addr
```

- Shows a brief summary of network interfaces and their addresses.
- Helps confirm the server is using the correct IP address and that interfaces are up.

```
[root@server ~]# ip -br addr
lo          UNKNOWN    127.0.0.1/8 ::1/128
eth0        UP         10.0.0.10/24 fe80::a00:27ff:fe12:3456/64
eth1        UP         192.168.1.10/24 fe80::5054:ff:feab:cd12/64
docker0     UP         172.17.0.1/16
veth1234    UP         172.17.0.2/16
```

43. ip route get 8.8.8.8 – Check Route to a Destination

```
ip route get 8.8.8.8
```

- Shows the selected route toward a sample destination.
- Helps verify routing, gateway and the network interface being used.

```
[root@server ~]# ip route get 8.8.8.8
8.8.8.8 via 192.168.1.1 dev eth0  src 192.168.1.10
cache
```

44. sudo nft list ruleset – Check Firewall Rules

```
sudo nft list ruleset
```

- Shows the current nftables firewall rules.
- Helps identify whether traffic is being allowed or blocked.

```
[root@server ~]# sudo nft list ruleset | head -20
table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;
        ct state established,related accept
        iif "lo" accept
        tcp dport 22 accept
        tcp dport 443 accept
        # other rules ...
    }
```

45. sudo tcpdump -ni any 'tcp port 443' – Observe Traffic

```
sudo tcpdump -ni any 'tcp port 443'
```

- Observes traffic reaching or leaving port 443.
- Helps determine if packets are arriving, being dropped, or leaving the server.

```
[root@server ~]# sudo tcpdump -ni any 'tcp port 443'
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux
cooked v2), snapshot length 262144 bytes

10:15:32.123456 eth0  In  IP 203.0.113.5.54321 >
10.0.0.10.443: Flags [S], seq 123456, win 64240,
length 0

10:15:32.123789 eth0  Out  IP 10.0.0.10.443 >
203.0.113.5.54321: Flags [S.], seq 987654, ack 123457,
win 65160, length 0
```

Senior Engineering Focus

- Differentiate between bind-address problems (listening only on localhost).
- Check firewall rules and network security groups.
- Verify routing and network interface configuration.
- Identify whether connections are blocked, dropped or rejected.
- Distinguish between application-level rejections and network-level failures.

Key Takeaways

- Verify the service is listening on the correct address and port.
- Check interface addresses and their status.
- Confirm routing and gateway configuration.
- Review firewall rules and cloud security group settings.
- Use tcpdump to see if traffic is reaching the server.
- Investigate application logs if connections are reaching the server but are still rejected.

VERIQTA

DNS Works Sometimes, but Production Requests Still Fail



Production Scenario

- Applications **intermittently fail to resolve** internal service names, causing cascading request timeouts.
- DNS appears to work sometimes, but production requests still fail.
- Use the commands below to investigate the real cause.

46. resolvectl status – Check systemd-resolved Configuration

```
resolvectl status
```

- Shows DNS servers, domains, search domains and current status.
- Helps verify which resolver is being used and its configuration.

```
[root@server ~]# resolvectl status
Global
       Protocols: +LLMNR +mDNS -DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: stub
Link 2 (eth0)
 Current Scopes: DNS
      DNS Servers: 10.0.0.10 10.0.0.11
       DNS Domain: example.internal
Link 3 (docker0)
 Current Scopes: none
```

47. getent ahosts api.example.com – Test Name Resolution

```
getent ahosts api.example.com
```

- Tests resolution through the system name-service configuration.
- Shows all resolved addresses (IPv4 and IPv6).

```
[root@server ~]# getent ahosts api.example.com
10.0.1.20       STREAM  api.example.com
10.0.1.20       DGRAM   api.example.com
10.0.1.20       RAW     api.example.com
10.0.1.21       STREAM  api.example.com
10.0.1.21       DGRAM   api.example.com
10.0.1.21       RAW     api.example.com
fd00::20        STREAM  api.example.com
fd00::20        DGRAM   api.example.com
fd00::20        RAW     api.example.com
```

48. dig api.example.com – Check DNS Response Records

```
dig api.example.com
```

- Shows detailed DNS response records.
- Helps verify the answer, TTL, and which DNS server responds.

```
[root@server ~]# dig api.example.com
; <<>> DiG 9.18.1 <<>> api.example.com
;; global options: +cmd
;; ANSWER SECTION:
api.example.com.      60    IN    A     10.0.1.20
api.example.com.      60    IN    A     10.0.1.21
;; Query time: 12 msec
;; SERVER: 10.0.0.10#53(10.0.0.10) (UDP)
;; WHEN: Wed Oct 21 12:34:56 UTC 2026
;; MSG SIZE  rcvd: 68
```

49. dig +trace example.com – Trace DNS Delegation

```
dig +trace example.com
```

- Traces the DNS delegation path from root to authoritative nameservers.
- Helps identify where resolution problems occur.

```
[root@server ~]# dig +trace example.com
.               518400  IN      NS      a.root-servers.net.
.               518400  IN      NS      b.root-servers.net.
.               518400  IN      NS      c.root-servers.net.
;; Received 239 bytes from 198.41.0.4#53(198.41.0.4) in 12 ms

com.            172800  IN      NS      a.gtld-servers.net.
com.            172800  IN      NS      b.gtld-servers.net.
com.            172800  IN      NS      c.gtld-servers.net.
;; Received 525 bytes from 192.5.6.30#53(192.5.6.30) in 18 ms

example.com.    86400   IN      NS      ns1.example.com.
example.com.    86400   IN      NS      ns2.example.com.
```

50. sudo tcpdump -ni any 'port 53' – Capture DNS Traffic

```
sudo tcpdump -ni any 'port 53'
```

- Captures DNS traffic in real time.
- Helps confirm if queries are sent, responses are received, or packets are dropped.

```
[root@server ~]# sudo tcpdump -ni any 'port 53'
tcpdump: data link type LINUX_SLL2
12:34:01.123456 eth0   IP 10.0.2.15.54321 > 10.0.0.10.53:
        12345+ A? api.example.com. (28)
12:34:01.123789 eth0   IP 10.0.0.10.53  > 10.0.2.15.54321:
        12345 1/0/0 A 10.0.1.20 (44)
12:34:05.567890 eth0   IP 10.0.2.15.54322 > 10.0.0.11.53:
        54321+ A? api.example.com. (28)
12:34:10.000123 eth0   IP 10.0.2.15.54322 > 10.0.0.11.53:
        54321+ A? api.example.com. (28)  [no response]
```

Senior Engineering Focus

- Investigate resolver differences, caching and DNS timeouts.
- Check for split-horizon DNS and service discovery issues.
- Determine whether the problem is in the application, the system resolver, the network, or the DNS server.
- Explain why a successful dig test does not prove that an application resolves names through the same path.

Important Operational Warning

- A successful dig or getent test only shows the system can resolve the name, not that the application uses the same resolver or network path.
- Check application configuration, container DNS settings, and network policies.
- Investigate DNS caching, timeouts and intermittent failures before making changes.

VERIQTA

Thousands of TCP Connections Are Stuck in TIME_WAIT



Production Scenario

- A reverse proxy experiences **intermittent outbound connection failures** during traffic spikes.
- Many TCP connections remain in TIME_WAIT.
- Use the commands below to investigate the real cause.

51. ss -tan state time-wait – Check TIME_WAIT Connections

```
ss -tan state time-wait
```

- Shows TCP connections in TIME_WAIT state.
- Helps determine if the number of TIME_WAIT connections is normal or unusually high.

```
[root@server ~]# ss -tan state time-wait | head -10
Recv-Q   Send-Q   Local Address:Port        Peer Address:Port
0        0        192.168.1.10:54321       10.0.0.5:443
0        0        192.168.1.10:54322       10.0.0.5:443
0        0        192.168.1.10:54323       10.0.0.5:443
0        0        192.168.1.10:54324       10.0.0.5:443
0        0        192.168.1.10:54325       10.0.0.5:443
0        0        192.168.1.10:54326       10.0.0.5:443
0        0        192.168.1.10:54327       10.0.0.5:443
0        0        192.168.1.10:54328       10.0.0.5:443
0        0        192.168.1.10:54329       10.0.0.5:443
0        0        192.168.1.10:54330       10.0.0.5:443
```

52. ss -tan state close-wait – Check CLOSE_WAIT Connections

```
ss -tan state close-wait
```

- Shows TCP connections in CLOSE_WAIT state.
- Helps identify connections that are waiting for the application to close them, which may indicate a resource leak.

```
[root@server ~]# ss -tan state close-wait | head -10
Recv-Q   Send-Q   Local Address:Port        Peer Address:Port
1        0        192.168.1.10:8080        10.0.0.25:51324
1        0        192.168.1.10:8080        10.0.0.25:51325
1        0        192.168.1.10:8080        10.0.0.25:51326
1        0        192.168.1.10:8080        10.0.0.25:51327
1        0        192.168.1.10:8080        10.0.0.25:51328
1        0        192.168.1.10:8080        10.0.0.25:51329
1        0        192.168.1.10:8080        10.0.0.25:51330
1        0        192.168.1.10:8080        10.0.0.25:51331
1        0        192.168.1.10:8080        10.0.0.25:51332
1        0        192.168.1.10:8080        10.0.0.25:51333
```

53. ss -tan state syn-sent – Check SYN_SENT Connections

```
ss -tan state syn-sent
```

- Shows TCP connections in SYN_SENT state.
- Helps identify incomplete outbound connection attempts, which may indicate network issues or unreachable services.

```
[root@server ~]# ss -tan state syn-sent | head -10
Recv-Q   Send-Q   Local Address:Port        Peer Address:Port
0        1        192.168.1.10:55678       10.0.0.50:443
0        1        192.168.1.10:55679       10.0.0.50:443
0        1        192.168.1.10:55680       10.0.0.50:443
0        1        192.168.1.10:55681       10.0.0.50:443
0        1        192.168.1.10:55682       10.0.0.50:443
0        1        192.168.1.10:55683       10.0.0.50:443
0        1        192.168.1.10:55684       10.0.0.50:443
0        1        192.168.1.10:55685       10.0.0.50:443
0        1        192.168.1.10:55686       10.0.0.50:443
0        1        192.168.1.10:55687       10.0.0.50:443
```

54. sysctl net.ipv4.ip_local_port_range – Check Ephemeral Port Range

```
sysctl net.ipv4.ip_local_port_range
```

- Shows the ephemeral port range used for outbound connections.
- Helps determine if the range is too small for the workload.

```
[root@server ~]# sysctl net.ipv4.ip_local_port_range
net.ipv4.ip_local_port_range = 32768  60999
[root@server ~]# cat /proc/sys/net/ipv4/ip_local_port_range
32768 60999
```

Key Information

- Ephemeral ports are used for outgoing connections.
- If the range is too small, the server can run out of available ports, causing connection failures.
- The default range is usually 32768 – 60999 on many systems.

Senior Engineering Focus

- Distinguish normal TCP lifecycle behaviour from port exhaustion, connection churn and application socket leaks.
- Investigate actual connection errors and application behaviour before changing TCP settings.
- Correlate with server logs, network devices, and upstream services.
- Check for misbehaving clients or unnecessary short-lived connections.

Production Insight

- TIME_WAIT is not inherently a failure.
- A high number of TIME_WAIT connections can be normal under high traffic.
- Investigate real connection errors and port allocation before changing TCP settings.
- Increasing limits may only postpone the problem if the root cause is a connection leak or excessive connection churn.

55. nstat -az – Check TCP/IP Protocol Counters

```
nstat -az
```

- Shows detailed TCP/IP protocol counters.
- Helps identify connection errors, retransmissions and other network issues.

```
[root@server ~]# nstat -az | head -10
#kernel    name --              value
           TcpInSegs            123456789
           TcpOutSegs           987654321      TcpInSegs
           TcpRetransSegs       12345          TcpInCsumErrors
           TcpInErrs            567            IpInReqdevss
           TcpInOdrdeErors      987654321      IpOutRevests
           IpInNoEpoos s        12
```

VERIQTA

Network Latency Is High, but Ping Looks Perfect



Production Scenario

- Users experience slow API responses despite healthy ICMP latency.
- Ping shows low latency, but real application traffic is slow.
- Use the commands below to investigate the real cause.

56. mtr -rwzc 20 api.example.com – Check Route and Latency

```
mtr -rwzc 20 api.example.com
```

- Shows route, packet loss and latency patterns.
- Helps identify where delays or packet loss occur along the path.

```
[root@server ~]# mtr -rwzc 20 api.example.com
Start: 2026-10-27T12:00:00+0000
HOST                 LOSS%  SNT  LAST   AVG  BEST  WRST  STDEV
1. 192.168.1.1       0.0%   20   0.3    0.4   0.2   0.8   0.2
2. 10.0.0.1          0.0%   20   1.1    1.3   0.8   3.1   0.6
3. 203.0.113.1       0.0%   20   5.2    5.8   4.9   9.1   1.2
4. 198.51.100.5      0.0%   20   22.4   30.1  18.2  80.3  15.6
5. 93.184.216.34     0.0%   20   21.8   24.7  20.1  33.6  4.1
```

57. tracepath api.example.com – Check Path and MTU

```
tracepath api.example.com
```

- Shows route, MTU sizes and path characteristics.
- Helps detect MTU issues or network path changes.

```
[root@server ~]# tracepath api.example.com
1?:  [LOCALHOST]      pmtu 1500
 1:  192.168.1.1        0.345ms
 2:  10.0.0.1           1.203ms
 3:  203.0.113.1        5.621ms
 4:  198.51.100.5      22.443ms
 5:  93.184.216.34     24.981ms    reached
 Resume: pmtu 1500  hops 5  back 5
```

58. curl ... – Check Detailed Request Timing

```
curl -sS -o /dev/null -w 'DNS:%{time_namelookup}
TCP:%{time_connect} TLS:%{time_appconnect}
TTFB:%{time_starttransfer} TOTAL:%{time_total}\n'
https://api.example.com
```

- Breaks down DNS, TCP, TLS and server response time.
- Helps identify which stage is causing the delay.

```
[root@server ~]# curl -sS -o /dev/null -w 'DNS:%{time_namelookup}\
TCP:%{time_connect} TLS:%{time_appconnect}\
TTFB:%{time_starttransfer} TOTAL:%{time_total}\n' https://api.example.com
DNS:0.012734 TCP:0.045821 TLS:0.113562
TTFB:1.240315  TOTAL:1.412432
```

59. ss -ti – Check TCP Connection Information

```
ss -ti
```

- Shows TCP connection details, congestion and performance info.
- Helps identify retransmissions, congestion or abnormal behaviour.

```
[root@server ~]# ss -ti | head -10
State   Recv-Q  Send-Q   Local Address:Port    Peer Address:Port
ESTAB   0       0        192.168.1.10:54321    93.184.216.34:443
        cubic wscale:7,7 rto:201 rtt:24.3/1.2 mss:1448
        rcv_space:14480 rcv_ssthresh:65535
        retrans:0/52 rcv_rtt:24.3 rcv_rttvar:1.2
        snd_cwnd:10 snd_ssthresh:65483 snd_wnd:28960
        pacing_rate 5.76Mbits/s delivery_rate 5.21Mbits/s
        bytes_acked:1256789 bytes_sent:987654
        segs_out:4321 segs_in:3987 data_segs_out:4210
        lost:0 retrans:2 dsack_dups:0 reord:0
```

60. ip -s link – Check Interface Errors and Drops

```
ip -s link
```

- Shows interface statistics, errors and packet drops.
- Helps identify network hardware or link issues.

```
[root@server ~]# ip -s link
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
   RX: bytes 1234567890  packets 987654   errors 0  dropped 12
   TX: bytes 987654321   packets 876543   errors 0  dropped 8
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
   RX: bytes 4567891230  packets 654321   errors 0  dropped 0
   TX: bytes 345678912   packets 543210   errors 0  dropped 0
```

Senior Engineering Focus

- Separate DNS delay, TCP connection time, TLS negotiation, server response time, and transport problems.
- Compare network path behaviour with application request timing.
- Look for packet loss, retransmissions, congestion, MTU issues and interface errors.
- Validate whether the problem is local, on the network path, or at the destination service.

Key Takeaways

- Ping only tests ICMP and does not measure application performance.
- Use multiple tools to check DNS, routing, TCP, TLS and server response time.
- Look for packet loss, retransmissions and interface errors.
- Compare results from different locations if possible.
- Identify the exact stage causing the delay before making changes.

VERIQTA

The Server Was Compromised, but Nothing Looks Obviously Wrong



Production Scenario

- Monitoring detects **unexpected outbound traffic** from a Linux production server.
- The server appears to be working normally.
- Use the commands below to investigate the real cause.

61. ss -tunap – Check Network Sockets and Owning Processes

```
ss -tunap
```

- Shows active TCP and UDP sockets and associated processes.
- Helps identify unexpected outbound connections.

```
[root@server ~]# ss -tunap
Netid   State    Recv-Q   Send-Q     Local Address:Port
                                     Peer Address:Port   Process
tcp     ESTAB    0        0          10.0.0.10:54321
93.184.216.34:443    users:(("curl",pid=2345,fd=5))
tcp     ESTAB    0        0          10.0.0.10:40218
185.199.110.133:80      users:(("python3",pid=4567,fd=3))
udp     UNCONN   0        0          0.0.0.0:68
0.0.0.0:*            users:(("dhclient",pid=789,fd=6))
```

62. ps auxwwf – Check Running Processes and Parent-Child Tree

```
ps auxwwf
```

- Shows running processes with full command lines and hierarchy.
- Helps identify suspicious or unexpected processes.

```
[root@server ~]# ps auxwwf
USER       PID  %CPU %MEM    VSZ   RSS TTY   STAT START   TIME COMMAND
root         1   0.0  0.1  16600  6200 ?     Ss   Oct26   0:01 /sbin/init
 ├─root   2345   0.1  0.3  23456 12340 ?     S    10:22   0:00 /usr/bin/python3
 │                                                            /opt/update.py
 ├─root   2350   0.0  0.2  21640  9656 ?     S    10:22   0:00 ├─ /usr/bin/curl
 │                                                              https://185.199.110.133/beacon
root     4567   0.0  0.1  19876  8432 ?     S    10:30   0:00 /usr/bin/bash -c
                                                               http://malicious.example/loader.sh
```

63. last -F – Check Login History

```
last -F
```

- Shows user login history with full date and time.
- Helps identify unauthorized or unusual logins.

```
[root@server ~]# last -F | head -10
user     tty      from            login@          logout@     duration
root     pts/1    203.0.113.45    Mon Oct 27 10:12   still logged in
ubuntu   pts/0    198.51.100.23   Mon Oct 27 08:41   09:10    00:28
root     pts/2    192.0.2.55      Mon Oct 27 02:15   02:20    00:05
deploy   pts/1    10.0.0.15       Oct 26 22:30       23:05    00:35
root     pts/0    45.77.12.8      Oct 26 21:14       21:14    00:00
```

64. journalctl -u ssh.service --since '24 hours ago' – Check SSH Events

```
journalctl -u ssh.service --since '24 hours ago'
```

- Shows recent SSH service events.
- Helps identify successful and failed login attempts.

```
[root@server ~]# journalctl -u ssh.service --since '24 hours ago' | head -10
Oct 27  08:41:12 server sshd[1023]: Accepted password for ubuntu
        from 198.51.100.23 port 49822 ssh2
Oct 27  10:12:05 server sshd[1456]: Accepted publickey for root
        from 203.0.113.45 port 55123 ssh2
Oct 27  12:04:18 server sshd[1890]: Failed password for invalid user
        admin from 45.77.12.8 port 40122 ssh2
Oct 27  12:10:31 server sshd[1901]: Received disconnect from 45.77.12.8
        port 40122:11:  Bye Bye [preauth]
```

65. systemctl list-timers --all – Check Systemd Timers

```
systemctl list-timers --all
```

- Shows all systemd timers, including inactive ones.
- Helps identify scheduled tasks that may trigger suspicious activity.

```
[root@server ~]# systemctl list-timers --all | head -10
NEXT                        LEFT       LAST                        PASSED  UNIT                ACTIVATES
Tue 2026-10-27 13:00:00     2h left    Tue 12:00:00                10min   logrotate.timer     logrotate.service
Tue 2026-10-27 14:30:00     3h left    Tue 11:30:00                40min   backup.timer        backup.service
Wed 2026-10-28 01:15:00     13h left   Tue 01:15:00                11h     update.timer        update.service
n/a                         n/a        Tue 10:22:11                1h      suspicious.timer    suspicious.service
n/a                         n/a        n/a                         n/a     malwarecheck.timer  malwarecheck.service
```

Senior Engineering Focus

- Correlate suspicious outbound connections with running processes and their parent processes.
- Review recent authentication activity for unauthorized access.
- Look for scheduled tasks, persistence mechanisms and unusual system behaviour.
- Distinguish between legitimate activity and malicious compromise.

Security Decision

- Preserve evidence and logs.
- Do not terminate suspicious processes immediately unless required for containment.
- Coordinate with your security and incident response team.
- Take backups or snapshots before making changes.

VERIQTA

Linux Permissions Look Correct, but Access Is Still Denied



Production Scenario

- A service **cannot access a directory** even though traditional Unix ownership and permissions appear correct.
- The issue may be related to ACLs, SELinux, AppArmor or service restrictions.
- Use the commands below to investigate the real cause.

66. namei -l /srv/app/data — Check Path Permissions

```
namei -l /srv/app/data
```

- Shows permissions for every component in the path.
- Helps identify a parent directory without execute (x) permission.

```
[root@server ~]# namei -l /srv/app/data
f: /srv/app/data
drwxr-xr-x     root     root     /
drwxr-xr-x     root     root     srv
drwxr-x---     deploy   deploy   app
drwxr-xr-x     deploy   deploy   data
```

67. getfacl /srv/app/data — Check Extended ACLs

```
getfacl /srv/app/data
```

- Shows extended access control lists (ACLs).
- Helps identify additional permission rules that affect access.

```
[root@server ~]# getfacl /srv/app/data
# file: srv/app/data
# owner: deploy
# group: deploy
user::rwx
user:backup:r-x
group::r-x
group:app:r--
mask::r-x
other::---
```

68. ls -Zd /srv/app/data — Check SELinux Labels

```
ls -Zd /srv/app/data
```

- Shows the SELinux security context for the file or directory.
- Helps identify label mismatches that can cause access denial.

```
[root@server ~]# ls -Zd /srv/app/data
drwxr-xr-x. deploy deploy unconfined_u:object_r:httpd_sys_content_t:s0
/srv/app/data
```

69. getenforce — Check SELinux Enforcement Mode

```
getenforce
```

- Shows the current SELinux enforcement status.
- Helps determine if SELinux is actively blocking access.

```
[root@server ~]# getenforce
Enforcing
(Possible values: Enforcing, Permissive, Disabled)
```

70. journalctl -k -g 'avc:|apparmor="DENIED"' --no-pager — Check MAC Denials

```
journalctl -k -g 'avc:|apparmor="DENIED"' --no-pager
```

- Searches kernel logs for SELinux (avc) or AppArmor denial records.
- Helps identify the exact reason access is being blocked.

```
[root@server ~]# journalctl -k -g 'avc:|apparmor="DENIED"' --no-pager
Oct 27 10:14:21 server kernel: avc:  denied  { read }
       for pid=1234 comm="nginx" name="data"
       dev="sda1" ino=123456 scontext=system_u:system_r:httpd_t:s0
       tcontext=unconfined_u:object_r:default_t:s0 tclass=dir

Oct 27 10:15:02 server kernel: audit: type=1400 apparmor="DENIED"
       operation="open" profile="/usr/sbin/nginx"
       name="/srv/app/data/config.yaml" pid=1234 comm="nginx"
       requested_mask="r" denied_mask="r" fsuid=1001 ouid=0
```

Senior Engineering Focus

- Investigate effective user and group identities.
- Check permissions on all path components.
- Examine ACLs, SELinux labels and AppArmor profiles.
- Review service configurations, container security contexts and sandbox restrictions.
- Identify which mechanism is actually denying access.

Key Takeaways

- A directory can be inaccessible if any parent directory lacks execute (x) permission.
- ACLs and SELinux/AppArmor can override traditional Unix permissions.
- Services may run with different identities or restricted capabilities.
- Always check logs for explicit denial messages.
- Verify the full path, effective user, groups and security context before making changes.

VERIQTA

The Linux Kernel Is Healthy Until Production Traffic Arrives



Production Scenario

- A server experiences **intermittent stalls** and **unexpected resets** during heavy network or storage activivity.
- The kernel appears healthy at low load.
- Use the commands below to investigate the real cause.

71. uname -a – Check Running Kernel

```
uname -a
```

- Shows the running kernel version, build and architecture.
- Helps verify if you are using an expected and supported kernel version.

```
[root@server ~]# uname -a
Linux server 6.8.12-200.fc40.x86_64 #1 SMP PREEMPT_DYNAMIC
Fri Sep 27 16:24:18 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
```

72. journalctl -k -b -p warning --no-pager – Check Kernel Warnings

```
journalctl -k -b -p warning --no-pager
```

- Shows kernel messages from the current boot with severity warning or higher.
- Helps identify hardware errors, driver issues and kernel problems.

```
[root@server ~]# journalctl -k -b -p warning --no-pager
Oct 27 08:14:32 server kernel: nvme 0000:01:00.0: I/O 1234
              queue 3 timeout,reset controller
Oct 27 08:14:32 server kernel: ata2.00: exception Emask 0x0
              SAct 0x0 SErr 0x400
Oct 27 08:15:01 server kernel: eth0: NIC Link is Down
Oct 27 08:15:04 server kernel: IPv6: rcu_sched detected
              stalls on CPUs/tasks
```

73. cat /proc/interrupts – Check Interrupt Distribution

```
cat /proc/interrupts
```

- Shows the number of hardware interrupts per CPU.
- Helps identify interrupt imbalance or a single CPU handling too many interrupts.

```
[root@server ~]# cat /proc/interrupts | head -10
           CPU0       CPU1       CPU2       CPU3
  0:     123456       1234        567        890   IO-APIC   2-edge      timer
  1:      98765       4321       7654        321   IO-APIC   1-edge      i8042
  8:     456789      12345       6789        456   IO-APIC   8-edge      nvme0q0
  9:     456781      12344       6788        455   IO-APIC   8-edge      nvme0q1
 16:    1234567      98765      43210       5678   IO-APIC  16-edge      eth0
 17:    1234560      98760      43200       5670   IO-APIC  17-edge      eth1
```

74. cat /proc/softirqs – Check Soft-Interrupt Activity

```
cat /proc/softirqs
```

- Shows soft-interrupt counts per CPU.
- Helps identify network, storage or other softirq pressure.

```
[root@server ~]# cat /proc/softirqs
                    CPU0       CPU1       CPU2       CPU3
HI:                   12         15         11          9
TIMER:            123456     123400     123410     123405
NET_TX:           987654     876543     765432     654321
NET_RX:          1234567     234567     345678     456789
BLOCK:             45678      56789      67890      78901
IRQ_POLL:              0          0          0          0
TASKLET:            1234       4321       2345       3456
SCHED:             98765      87654      76543      65432
HRTIMER:           54321      43210      32109      21098
RCU:              123456     123456     123456     123456
```

75. cat /proc/pressure/cpu – Check CPU Pressure

```
cat /proc/pressure/cpu
```

- Shows CPU pressure stall information (PSI).
- Helps detect contention between application work and kernel processing.

```
[root@server ~]# cat /proc/pressure/cpu
some avg10=0.12 avg60=0.05 avg300=0.02  total=12345678
full avg10=0.03 avg60=0.01 avg300=0.00  total=3456789
```

Senior Engineering Focus

- Investigate kernel warnings and hardware or driver errors.
- Check for interrupt imbalance and high softirq activity.
- Analyze CPU pressure and resource contention.
- Correlate kernel events with application, network and storage activity.
- Determine whether the issue is caused by hardware, drivers, configuration or excessive load.

Key Takeaways

- A healthy kernel at low load does not guarantee stability under production traffic.
- Interrupt and softirq imbalance can cause performance problems and stalls.
- Use kernel logs and pressure metrics to identify the real bottleneck.
- Investigate and fix the underlying cause rather than only

VERIQTA

A Container Is OOMKilled While the Linux Host Looks Healthy



Production Scenario

- A containerized application **repeatedly terminates (OOMKilled).**
- The Linux host appears to have adequate available memory.
- Use the commands below to investigate the real cause.

76. systemd-cgls – Check cgroup Hierarchy

`systemd-cgls`

- Shows the cgroup hierarchy and running processes.
- Helps locate the container's cgroup and verify its place in the system hierarchy.

```
[root@server ~]# systemd-cgls | head -20
Control group /:
├─docker.service
│   ├─docker-3f2a1c8d4e5f8b2c9d0e7f6a123456789abcdef.scope
│   │   ├─12345 /usr/bin/containerd
│   │   └─12367 /usr/bin/myapp
├─kubepods.slice
│   └─kubepods-besteffort.slice
│       └─kubepods-besteffort-pod12345678_abcdef12.scope
│           ├─ 45678 /pause
└──         └─ 45679 /usr/bin/app
```

77. systemd-cgtop – Check cgroup Resource Usage

`systemd-cgtop`

- Shows real-time resource consumption across cgroups.
- Helps identify which container or service is using memory, CPU or I/O resources.

```
[root@server ~]# systemd-cgtop
Control Group          Tasks   %CPU   Memory  Input/s  Output/s
/                        352    5.2     1.2G     1.1M      980K
/docker:service           85    2.1     512M     450K      320K
 /docker-3f2a1c8d         12    1.8     480M     400K      280K
/kubepods.slice           96    3.0     600M     620K      540K
/system.slice             54    0.5     120M     100K       80K
/user:slice                8    0.1      20M      10K       10K
```

78. cat /sys/fs/cgroup/memory.max – Check Memory Limit

`cat /sys/fs/cgroup/memory.max`

- Shows the memory limit for the cgroup at the specified path.
- Helps confirm the container's actual memory limit.

```
[root@server ~]# cat /sys/fs/cgroup/memory.max
1073741824
[root@server ~]# numfmt --to=iec 1073741824
1.0G
```

79. cat /sys/fs/cgroup/memory.current – Check Current Usage

`cat /sys/fs/cgroup/memory.current`

- Shows the current memory usage for the cgroup at the specified path.
- Helps determine if the container is close to or exceeding its limit.

```
[root@server ~]# cat /sys/fs/cgroup/memory.current
956301312
[root@server ~]# numfmt --to=iec 956301312
912M
```

80. cat /sys/fs/cgroup/memory.stat – Check Memory Accounting

`cat /sys/fs/cgroup/memory.stat`

- Shows detailed memory accounting for the cgroup.
- Helps analyze which memory categories are being used.

```
[root@server ~]# cat /sys/fs/cgroup/memory.stat | head -10
anon                743178240
file                12390400
kernel              15667200
kernel_stack        2097152
pagetables          4194304
shmem               8388608
file_mapped         0
file_dirty          0
file_writeback      0
swapcached          0
```

Senior Engineering Focus

- Understand cgroup v2 memory enforcement.
- Investigate container resource limits and memory usage.
- Correlate container OOM events with cgroup metrics.
- Analyze memory accounting categories (anon, file, kernel, etc).
- Determine whether the issue is a memory leak, a legitimate high-memory workload, or an incorrectly configured limit.

Important Distinction

- Commands 78 to 80 inspect the cgroup at the specified path.
- For an affected container, identify and inspect its actual cgroup directory rather than assuming the root cgroup represents the container.
- Container cgroup paths typically differ by runtime (e.g., Docker, containerd, Kubernetes) and may include container IDs.
- Always verify the correct cgroup path before interpreting the results.

VERIQTA

The Server Is Slow, but Every Dashboard Shows Green



Production Scenario

- Customers report slow transactions, but aggregate CPU, memory and disk utilization remain below alert thresholds.
- The system looks healthy on dashboards, but performance is poor.

81. cat /proc/pressure/cpu — Examine CPU pressure

```
cat /proc/pressure/cpu
```

- Shows CPU pressure stall information (PSI).
- Helps identify when CPU is under pressure, even if utilization looks low.

```
[root@server ~]# cat /proc/pressure/cpu
some   avg10=0.42  avg60=0.28  avg300=0.12  total=12345678
full   avg10=0.05  avg60=0.03  avg300=0.01  total=3456789
```

82. cat /proc/pressure/memory — Examine memory pressure

```
cat /proc/pressure/memory
```

- Shows memory pressure stall information (PSI).
- Helps detect memory contention and reclaim activity.

```
[root@server ~]# cat /proc/pressure/memory
some   avg10=0.31  avg60=0.19  avg300=0.08  total=8765432
full   avg10=0.02  avg60=0.01  avg300=0.00  total=1234567
```

83. cat /proc/pressure/io — Examine I/O pressure

```
cat /proc/pressure/io
```

- Shows I/O pressure stall information (PSI).
- Helps identify disk or storage bottlenecks.

```
[root@server ~]# cat /proc/pressure/io
some   avg10=0.27  avg60=0.16  avg300=0.07  total=6543210
full   avg10=0.01  avg60=0.00  avg300=0.00  total=987654
```

84. sar -q 1 5 — Inspect run queue and load trends

```
sar -q 1 5
```

- Shows run queue length and load averages over time.
- Helps detect scheduling delays and CPU contention.

```
[root@server ~]# sar -q 1 5
12:00:01      RUNQ    LDAVG-1   LDAVG-5   LDAVG-15
12:00:01        2       0.43      0.38       0.32
12:00:02        3       0.56      0.42       0.34
12:00:03        4       0.72      0.51       0.40
12:00:04        6       0.88      0.63       0.47
12:00:05        5       0.76      0.58       0.43
```

85. pidstat -w 1 5 — Examine process context-switch activity

```
pidstat -w 1 5
```

- Shows context switches, voluntary and non-voluntary.
- Helps identify high process switching and scheduling pressure.

```
[root@server ~]# pidstat -w 1 5
Linux 6.8.12  (server)    10/27/2026   _x86_64_   (4 CPU)

12:00:01   UID   PID   cswch/s  nvcswch/s  nBLOCK/s  majflt/s  minflt/s
12:00:01  1000  2345   1450.23    1020.17     12.34      0.00     32.56
12:00:02  1000  2345   1623.45    1187.32     18.76      0.00     41.23
12:00:02  1000  2345   1897.21    1356.77     22.10      0.00     52.67
12:00:04  1000  2345   1643.07    1204.33     17.42      0.00     46.31
12:00:05  1000  2345   1489.56    1102.18     13.87      0.00     38.90
```

Senior Engineering Focus

- Explain why average utilization can hide saturation, scheduling delays and resource pressure.
- Look beyond CPU, memory and disk percentages.
- Correlate PSI, run queue, context switches and latency to find the real issue.

Production Insight

- Resource pressure and request latency can reveal failures that simple utilization percentages miss.
- A healthy dashboard does not always mean a healthy system.

VERIQTA

A Deployment Changed Nothing, Yet Production Started Failing



Production Scenario

- A previously healthy application becomes **unstable without a new application release.**
- The issue may be caused by changes in the environment, packages, configuration, infrastructure or dependencies.
- Use the commands below to investigate the real cause.

86. journalctl --since '2 hours ago' --no-pager – System Events

```
journalctl --since '2 hours ago' --no-pager
```

- Shows system events and logs from the specified time.
- Helps build a timeline of changes, errors and warnings.

```
[root@server ~]# journalctl --since '2 hours ago' --no-pager
Oct 27 08:12:01 server systemd[1]: Started nginx.service.
Oct 27 08:14:33 server kernel: nic: link is down
Oct 27 08:15:10 server CRON[1234]: (root) CMD
Oct 27 08:16:27 server systemd[1]: Failed to start
                       database.service.
Oct 27 08:20:41 server NetworkManager[987]:  device eth0:
                       carrier lost
```

87. systemctl list-units --state=failed – Check Failed Units

```
systemctl list-units --state=failed
```

- Shows systemd units that are in a failed state.
- Helps identify services that failed after the issue started.

```
[root@server ~]# systemctl list-units --state=failed
UNIT                LOAD    ACTIVE   SUB      DESCRIPTION
nginx.service       loaded  failed   failed   A high performance
database.service    loaded  failed   failed   Database Service
redis.service       loaded  failed   failed   Redis In-Memory DB
backup.timer        loaded  failed   failed   backup Timer
app-worker.service  loaded  failed   failed   Application Worker
5 loaded units listed.
```

88. rpm -qa --last – Check Recent RPM Installations

```
rpm -qa --last
```

- Shows recently installed RPM packages (newest first).
- Helps identify system or dependency changes on RPM-based systems (e.g. RHEL, CentOS, Rocky Linux, AlmaLinux).

```
[root@server ~]# rpm -qa --last | head -10
kernel-5.14.0-503.16.1.el9.x86_64     Tue Oct 27 07:10:22 2026
openssl-3.0.7-18.el9.x86_64           Tue Oct 27 06:58:11 2026
nginx-1.22.1-9.el9.x86_64             Tue Oct 27 06:55:03 2026
glibc-2.34-83.el9.x86_64              Tue Oct 27 05:20:14 2026
bash-5.1.16-6.el9.x86_64              Mon Oct 26 23:11:36 2026
systemd-252-29.el9.x86_64             Mon Oct 26 22:47:18 2026
tzdata-2026a-1.el9.noarch             Mon Oct 26 21:10:02 2026
curl-7.76.1-26.el9.x86_64             Mon Oct 26 20:55:49 2026
ca-certificates-2026.2.1-1.el9        Mon Oct 26 20:40:27 2026
vim-enhanced-9.0.123-1.el9            Mon Oct 26 19:32:10 2026
```

89. find /etc -xdev -type f -mmin -120 -ls – Recent Config Changes

```
find /etc -xdev -type f -mmin -120 -ls
```

- Shows files in /etc modified in the last 120 minutes.
- Helps identify recent configuration changes that may have affected the application.

```
[root@server ~]# find /etc -xdev -type f -mmin -120 -ls
131108    4 -rw-r--r--    1 root root        1234
          Oct 27 08:10 /etc/nginx/nginx.conf
131225    8 -rw-------    1 root root        2981
          Oct 27 08:05 /etc/ssh/sshd_config
131300    4 -rw-r--r--    1 root root         567
          Oct 27 07:58 /etc/cron.d/backup
131402    4 -rw-r--r--    1 root root        1024
          Oct 27 07:47 /etc/sysctl.conf
131410    4 -rw-r--r--    1 root root        2048
          Oct 27 07:35 /etc/redis/redis.conf
```

90. timedatectl status – Check System Time

```
timedatectl status
```

- Shows current time, time zone and synchronization status.
- Helps verify that the system time is correct.

```
[root@server ~]# timedatectl status
               Local time: Tue 2026-10-27 08:22:14 UTC
           Universal time: Tue 2026-10-27 08:22:14 UTC
                 RTC time: Tue 2026-10-27 08:22:14
                Time zone: Etc/UTC (UTC, +0000)
System clock synchronized: yes
              NTP service: active
          RTC in local TZ: no
```

Senior Engineering Focus

- Investigate environmental drift and system changes.
- Check for automated package updates or configuration changes.
- Look for expiring credentials, certificates or keys.
- Identify infrastructure and dependency failures.
- Correlate system events with the time the issue started.

Incident Lesson

- Absence of an application deployment does not mean the production environment remained unchanged.
- System updates, configuration changes, infrastructure events or expired components can cause failures.
- Always investigate the full environment, not just the application code.

VERIQTA

Production Logs Are Missing During the Exact Incident Window



Production Scenario

- An outage occurs, but the logs needed for investigation are incomplete or unavailable.
- Logs may be missing due to retention limits, storage issues, corruption or rate limiting.
- Use the commands below to investigate the real cause.

91. journalctl --list-boots – Identify Available Boot Journals

```
journalctl --list-boots
```

- Shows all available boot entries in the journal.
- Helps determine if logs exist for the incident time period.

```
[root@server ~]# journalctl --list-boots
INDEX   BOOT ID          FIRST ENTRY                 LAST ENTRY
0       8a3d2f6c9e4b4d1a  Mon 2026-10-26 08:11:22  Mon 2026-10-26 16:45:10
1       c4b8e9a7d1234567  Tue 2026-10-27 02:14:05  Tue 2026-10-27 10:32:18
2       f9d1c2e4b67890ab  Tue 2026-10-27 10:45:33  Tue 2026-10-27 16:20:01
```

92. journalctl --disk-usage – Inspect Journal Storage Usage

```
journalctl --disk-usage
```

- Shows how much disk space is used by journal files.
- Helps identify if logs are being rotated or removed due to resource limits.

```
[root@server ~]# journalctl --disk-usage
Archived and active journals take up 512.0M in the file system.
Reserved space for future journals: 128.0M
Total disk usage:                   640.0M
```

93. journalctl --verify – Check Journal File Integrity

```
journalctl --verify
```

- Verifies the integrity of journal files.
- Helps detect corrupted or invalid log entries.

```
[root@server ~]# journalctl --verify
Verifying archived journal /var/log/journal/8a3d2f6c9e4b4d1a/system.journal ...
PASS
Verifying archived journal /var/log/journal/c4b8e9a7d1234567/system.journal ...
PASS
Verifying archived journal /var/log/journal/f9d1c2e4b67890ab/system.journal ...
File header invalid, file is corrupt: /var/log/journal/f9d1c2e4b67890ab/system.journal
```

94. systemctl status systemd-journald – Check Service Health

```
systemctl status systemd-journald
```

- Shows the status of the journald service.
- Helps detect configuration issues, failures or rate limiting.

```
[root@server ~]# systemctl status systemd-journald
● systemd-journald.service - Journal Service
     Loaded: loaded (/usr/lib/systemd/system/systemd-journald.service;
             enabled; preset: enabled)
     Active: active (running) since Tue 2026-10-27 08:11:20 UTC
       Docs: man:systemd-journald.service(8)
   Main PID: 312 (systemd-journald)
     Tasks: 1 (limit: 4915)
     Memory: 24.3M
        CPU: 1.251s
     CGroup: /system.slice/systemd-journald.service
             └─312 /usr/lib/systemd/systemd-journald
```

95. journalctl -u systemd-journald --since '24 hours ago' --no-pager – Review Journald Events

```
journalctl -u systemd-journald --since '24 hours ago' --no-pager
```

- Shows recent events for the journald service.
- Helps identify issues such as rate limiting, storage limits or errors.

```
[root@server ~]# journalctl -u systemd-journald \
                  --since '24 hours ago' --no-pager
Oct 26 09:12:41 server systemd-journald[312]: Journal started
Oct 26 14:22:10 server systemd-journald[312]: Rotating journal files
Oct 26 16:55:03 server systemd-journald[312]: Vacuuming done,
                      freed 256.0M of archived journals
Oct 27 07:48:21 server systemd-journald[312]: Rate limit reached,
                      suppressing 120 messages
Oct 27 08:13:05 server systemd-journald[312]: Failed to write entry:
                      No space left on device
```

Senior Engineering Focus

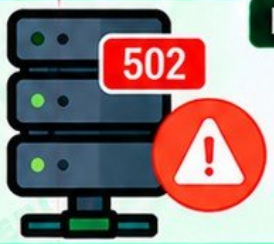
- Investigate volatile logging and retention limits.
- Check for journal corruption or missing log segments.
- Look for rate limiting, storage constraints or configuration issues.
- Verify whether logs are being forwarded to a centralized logging system.
- Correlate journald events with the incident timeline.

Architecture Decision

- Define log retention requirements based on your incident investigation needs.
- Implement remote log collection and centralized storage.
- Ensure sufficient storage capacity and proper log rotation.
- Configure alerting for log collection failures.
- Establish incident evidence requirements before a failure occurs.

VERIQTA

Final Production War Room: Diagnose a Multi-Layer Linux Outage



Production Scenario

- A Linux-hosted API begins returning HTTP 502 errors after a traffic spike.
- CPU utilization rises, request latency increases, file descriptors approach their limit, and database connections begin timing out.
- Multiple symptoms appear simultaneously.
- The challenge is determining which are root causes and which are consequences.

96. date -u – Establish a consistent incident timeline

```
date -u
```

- Shows the current UTC time.
- Helps create a consistent timeline across systems and logs.

```
[root@server ~]# date -u
Mon Oct 27 14:32:18 UTC 2026
```

97. systemctl --failed – Identify failed system services

```
systemctl --failed
```

- Shows systemd units that are in a failed state.
- Helps identify service failures that may be causing the outage.

```
[root@server ~]# systemctl --failed
UNIT            LOAD   ACTIVE SUB    DESCRIPTION
api.service     loaded failed failed API Application Service
db.service      loaded failed failed Database Service
cache.service   loaded failed failed Cache Service
3 loaded units listed.
```

98. ss -s – Examine connection-state pressure

```
ss -s
```

- Shows a summary of socket states.
- Helps identify high connection counts, TIME_WAIT, or other network pressure.

```
[root@server ~]# ss -s
Total: 89043
TCP:   75621 (estab 42103, closed 21043, orphaned 0,
       timewait 31245)
Transport Total     IP      IPv6
*         89043     -       -
RAW:      0         0       0
UDP:      3123      2890    233
TCP:      75621     67321   8300
INET:     78744     70211   8533
FRAG:     0         0       0
```

99. vmstat 1 10 – Correlate CPU, memory, scheduling and I/O

```
vmstat 1 10
```

- Shows CPU, memory, scheduling and I/O statistics at 1 second intervals.
- Helps identify resource contention during the incident.

```
[root@server ~]# vmstat 1 10
procs -----------memory---------- ---swap-- -----io---- -system-- ----cpu----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa
12  4      0 524288  81232 204832    0    0 10240 20480 4567 8921 65 20 10  5
15  6      0 510720  81200 204100    0    0 12032 29001 5011 10233 72 18  6  4
18  8      0 498112  81012 203900    0    0 15023 32011 6233 11002 78 16  4  2
20  7      0 492110  80944 203001    0    0 18901 41022 7021 12033 81 14  3  2
16  5      0 501220  80911 203330    0    0 14021 31002 6100 11520 74 18  5  3
```

100. journalctl -p err --since '30 minutes ago' --no-pager – Review recent system errors

```
journalctl -p err --since '30 minutes ago' --no-pager
```

- Shows recent system error and higher severity log entries.
- Helps identify the immediate cause and related failures.

```
[root@server ~]# journalctl -p err --since '30 minutes ago' --no-pager
Oct 27 14:05:21 server kernel: TCP: possible SYN flooding on port 443
Oct 27 14:06:10 server api[2345]: ERROR: failed to connect to database
Oct 27 14:07:32 server nginx[1023]: upstream timed out (110: Connection timed out)
Oct 27 14:08:11 server systemd[1]: db.service: Main process exited, code=exited, status=1/FAILURE
```

Final Senior Engineering Exercise

1. Establish customer impact and the incident timeline.
2. Identify the most likely initial failure domain.
4. Correlate process, network, resource and application evidence.
5. Distinguish symptoms from root causes.
6. Choose a low-risk mitigation.
7. Verify recovery using application health and customer-facing signals.
8. Recommend preventive engineering changes.

Key Takeaways

- Multiple symptoms can occur at the same time, but not all are root causes.
- Use a structured approach and objective evidence to find the initial failure.
- Correlate logs, system metrics, network data and application behavior.
- Mitigate carefully and verify recovery using real user and application signals.
- Implement improvements to prevent similar incidents in the future.

Production Insight

- A traffic spike can expose underlying capacity limits, configuration issues or dependency failures.
- Use system data to separate root causes from cascading effects.
- A successful investigation should lead to both recovery and long-term engineering improvements.